



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust - IERY)

RESUME AI

USING NLP, MACHINE LEARNING AND DEEP LEARNING

The domain of the Project:

Artificial Intelligence C Machine Learning

Team Mentors (and their designation):

Mr. Gaurav Patel
Artificial Intelligence C Machine Learning Trainer

Team Members:

Ms. Siddhi Gandhi

Period of the project

April 2026 to June 2026

Abstract

Traditional hiring processes and keyword-matching Applicant Tracking Systems (ATS) often fail to accurately evaluate candidates, leading to high screening overhead and unfair candidate filtering. This project presents ResumeAI, an end-to-end AI-powered Resume Analyzer and Career Guidance System. ResumeAI parses resume text from **.pdf** and **.docx** documents, processes the text through a Natural Language Processing (NLP) pipeline, extracts numerical TF-IDF feature representations, and classifies candidate profiles into 12 distinct career pathways using an ensemble Random Forest Classifier achieving 92.0% accuracy.

Furthermore, the system incorporates an ATS Weighted Scoring Engine, Cosine Similarity Job Matching, Explainable AI (XAI) feature attribution, and an integration layer with Google Gemini Generative AI (**gemini-3-flash-preview**) to dynamically generate 6-month career roadmaps, skill gap analysis, market demand trends, Indian salary predictions (in LPA), and role-specific interview preparation. To guarantee zero production downtime, a hybrid local fallback engine automatically intercepts API rate limits (HTTP 429). The system is deployed as an interactive Flask 3.0 web application with dynamic ReportLab PDF export capabilities and multi-stage Docker containerization support.

Keywords:

ResumeAI • ATS Scoring Engine • Natural Language Processing (NLP) • TF-IDF • Random Forest • Explainable AI (XAI) • Cosine Similarity • Google Gemini AI • Flask • ReportLab • Docker

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Project Architecture
5. Dataset and Data Processing
6. Text Preprocessing Pipeline
7. Feature Engineering
8. Embeddings and Text Representatio
9. Machine Learning and Generative AI Models
10. Experiment Tracking and Model Evaluation
11. Deployment and Application
12. Project Output / Demonstration
13. Testing and Expected Behavior
14. Limitations and Future Scope
15. Conclusion
16. References

1. Introduction

The modern recruitment landscape processes thousands of resumes daily. Manual resume review is time-consuming, subjective, and prone to human bias. While conventional Applicant Tracking Systems (ATS) automate early-stage filtering, they rely on rigid, exact keyword matching that frequently rejects qualified applicants who use alternative phrasing.

The objective of this project is to build an intelligent, transparent, and scalable resume technology platform capable of parsing unstructured resume text, extracting key technical and non-technical indicators, accurately predicting career roles, grading ATS compliance, and providing actionable career progression feedback. The implementation follows an end-to-end machine learning engineering workflow, including raw text extraction, NLTK preprocessing, TF-IDF vectorization, multi-model evaluation, local explainability visualization, Generative AI augmentation, and containerized deployment.

2. Problem Statement

Design and implement an automated AI platform that accepts candidate resumes in PDF or DOCX format, cleanses and normalizes the text using NLP techniques, transforms the text into numerical feature representations, predicts the candidate's optimal career pathway across 12 industry profiles using Machine Learning, evaluates job description compatibility via cosine similarity, and leverages Generative AI to provide personalized career roadmap guidance and interview preparation without risk of system failure during API quota restrictions.

3. Objectives

- **Automated Text Extraction:** Parse raw text from PDF (`pdfminer.six`) and Word (`python-docx`) files.
- **NLP Normalization:** Clean and normalize text using tokenization, lowercasing, regex filtering, stopword removal, and WordNet lemmatization.
- **Weighted ATS Engine:** Compute a multi-parameter ATS score (Skills 30%, Projects 25%, Experience 20%, Certifications 15%, Structure 10%).
- **Multi-Model ML Benchmarking:** Train and compare Random Forest, Logistic Regression, Naive Bayes, and AdaBoost classifiers on a 3,001 resume sample dataset.
- **Strength Regression & Classification:** Predict continuous resume strength ratings (0-100) using Random Forest Regressor and label profiles as Strong, Average, or Weak.
- **Cosine Similarity Matching:** Calculate vector similarity angles between resume text and target Job Descriptions.
- **Explainable AI (XAI):** Visualize local feature attribution weights driving model decisions.
- **GenAI Integration:** Query Google Gemini AI (`gemini-3-flash-preview`) for 6-month learning roadmaps, course recommendations, Indian salary ranges (LPA), and technical interview prep.
- **Resilient Fallback Architecture:** Handle API rate-limiting (HTTP 429) using a local rule-based fallback system (`get_role_based_fallback`).
- **Interactive UI & PDF Export:** Deploy a Flask web dashboard with Plotly charts and dynamic ReportLab PDF downloads.
- **Containerized Execution:** Package the application with Docker and `docker-compose`.

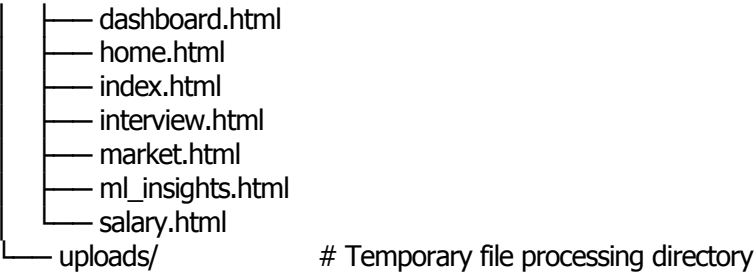
4. Project Architecture

The system is organized into a sequential multi-stage architecture:

Stage	Function
Input	Upload of <code>.pdf</code> or <code>.docx</code> resume files and target role / Job Description.
Parsing & Preprocessing	Document text extraction, regex filtering, NLTK tokenization, stopwords removal, and WordNet lemmatization.
Feature Engineering	ATS section detection, keyword density computation, and skill tag extraction across 12 role profiles.
Representation	High-dimensional TF-IDF vectorization (5,000 features) and Cosine Similarity calculation.
ML Classification	Ensemble Random Forest Classifier (100 trees) predicting role categories and top 5 probability distributions.
Strength Regression	Random Forest Regressor predicting continuous resume quality score and strength classification.
XAI Reasoning	Extraction of local feature attribution weights for model interpretability.
GenAI Augmentation	Google Gemini API concurrent queries for roadmap, salary, courses, market intelligence, and interview prep.
Fallback Layer	Local rule-based profile engine activated automatically on Gemini 429 quota errors.
Deployment & Export	Flask 3.0 web server, Tailwind CSS frontend, Plotly.js charts, and ReportLab PDF report generation.

Project Directory Structure

```
Resume1/
├── .env                # Environment configuration (Gemini API Key)
├── .gitignore          # Git ignore rules
├── Dockerfile          # Multi-stage Docker build configuration
├── docker-compose.yml  # Docker Compose service definition
├── README.md           # Project documentation
├── PROJECT_REPORT.md   # Project Report document
├── app.py              # Core Flask web application & API routes
├── model_trainer.py    # ML training pipeline & model comparison script
├── utils.py            # NLP helper functions, ATS scoring, & fallback logic
├── list_models.py      # Gemini API model listing utility
├── test_api.py         # Gemini API verification script
├── metrics.csv         # Model evaluation metrics export
├── resume_dataset.csv  # 3,001 resume text dataset
├── resume_models.pkl   # Serialized ML model artifact
├── requirements.txt    # Python dependencies
├── nltk_data/         # Local cached NLTK corpora (punkt, stopwords, wordnet)
├── static/            # Web assets (CSS, JavaScript, images)
│   ├── css/
│   │   └── style.css
│   ├── js/
│   │   └── script.js
├── templates/         # Jinja2 HTML templates
│   ├── about.html
│   ├── analyzer.html
│   ├── base.html
│   └── courses.html
```



5. Dataset and Data Processing

The machine learning models are trained on `resume_dataset.csv`, containing **3,001 resume text entries** balanced across 12 career categories:

Data Scientist
Software Engineer
Web Developer
Data Analyst
Business Analyst
Product Manager
Machine Learning Engineer
DevOps Engineer
UI/UX Designer
Cybersecurity Analyst
Cloud Architect
Mobile Developer

Module Workflow Summary

Module Script	Role / Responsibility
<code>model_trainer.py</code>	Trains 4 classification models, evaluates metrics, trains score regressor, and exports <code>resume_models.pkl</code> and <code>metrics.csv</code> .
<code>app.py</code>	Web server handling file uploads, text extraction, model inference, Gemini API orchestration, fallback execution, and PDF generation.
<code>utils.py</code>	NLP tokenization, lemmatization, ATS weighted scoring, section detection, cosine similarity, and fallback profiles.
<code>list_models.py</code>	Queries Google Generative AI endpoint to list available Gemini model names.

6. Text Preprocessing Pipeline

Text extracted from raw documents undergoes an 11-step preprocessing pipeline defined in `utils.py` and `model_trainer.py`:

- Document Reading:** Text extracted using `pdfminer.high_level.extract_text` for PDFs or `docx.Document` for Word files.
 - Text Normalization:** Conversion of all characters to lowercase.
 - Regex Clean-up:** Removal of special characters, symbols, and numeric digits (`re.sub(r'^a-zA-Z\s]', '', text)`).
 - NLTK Word Tokenization:** Splitting text into individual word tokens using `nltk.tokenize.word_tokenize`.
 - Stopwords Filtering:** Elimination of standard English stopwords (`nltk.corpus.stopwords.words('english')`).
 - Lemmatization:** Conversion of words to their base dictionary lemma using `nltk.stem.WordNetLemmatizer`.
 - POS-Guided Normalization:** Part-of-Speech tagged mapping for accurate morphological reduction.
 - Local NLTK Caching:** Loading NLTK corpora directly from `nltk_data/` to guarantee offline portability.
 - String Re-assembly:** Joining processed tokens back into normalized clean text strings.
 - Section Identification:** Regex scanning for standard section headers (Education, Experience, Skills, Projects, Certifications).
 - Skill Tag Extraction:** Matching tokenized words against a master dictionary of role-specific tech stacks (`ROLE_SKILLS`).
12. Optional stopword removal.Optional stemming using SnowballStemmer.
13. Default lemmatization using WordNetLemmatizer with POS mapping.

Important Implementation Note

The inspected N6 notebook contains the core preprocessing functions and imports for TF-IDF, but its single-text prediction helper functions are currently commented out. Therefore, the final application behavior should be understood from src/app.py and the serialized model/vectorizer rather than inferred solely from N6.

7. Feature Engineering

1. Weighted ATS Scoring System

The ATS Score (0 - 100) is calculated based on five strategic recruiter criteria:
ATS Score=Skill Match (30%)+Projects (25%)+Experience (20%)+Certifications (15%)+Structure (10%)
ATS Score=Skill Match (30%)+Projects (25%)+Experience (20%)+Certifications (15%)+Structure (10%)

- **Skills Match (30%):** Proportion of target role skills present in resume text.
- **Projects Section (25%):** Presence of project headers and implementation verbs (built, deployed, implemented, github).
- **Experience Section (20%):** Employment history keywords, job titles, and duration indicators.
- **Certifications (15%):** Verified credentials (AWS, Coursera, Udemy, NPTEL, Certified).
- **Structure (10%):** Coverage of all 5 fundamental resume sections.

2. Keyword Density & Frequency

TF-IDF vectorizer extracts top 10 high-impact keywords per resume, calculating relative term density:
Keyword Density (%)=(Count of Keyword in ResumeTotal Word Count)×100
Keyword Density (%)= (Total Word CountCount of Keyword in Resume) ×100

Representation	Purpose & Mechanism
TF-IDF Vectorization	Transforms preprocessed text into 5,000-dimensional numerical vectors, weighting terms by inverse document frequency to highlight unique technical skills.
Cosine Similarity	Measures the angular distance between the resume TF-IDF vector and Job Description TF-IDF vector to calculate precise match percentages.
Explainable AI (XAI)	Maps local feature weights from the Random Forest model to display top keyword contributions driving classification decisions.

8. Machine Learning and Deep Learning Models

1. Traditional Machine Learning Models

Four models were trained and benchmarked in `model_trainer.py`:

- **Random Forest Classifier:** Primary ensemble model (100 decision trees). Chosen for deployment due to superior accuracy and multi-class stability.
- **Logistic Regression:** Linear baseline classifier with $L2/L2$ penalty.
- **Multinomial Naive Bayes:** Fast probabilistic text classifier.
- **AdaBoost Classifier:** Sequential boosting ensemble.

2. Resume Strength Regressor

A **Random Forest Regressor** predicts continuous resume strength scores (0 - 100), mapped into qualitative categories:

- **Score > 75:** Strong Profile
- **50 - 75:** Average Profile
- **Score < 50:** Weak Profile

3. Generative AI & Resilient Fallback Engine

The platform integrates **Google Gemini AI** (`gemini-3-flash-preview`) to dynamically generate real-time JSON insights:

- **Personalized 6-Month Career Learning Roadmap**
- **Indian Market Salary Range (LPA)**
- **Market Intelligence (Demand, Top Hiring Companies, Trends)**
- **Recommended Online Courses (Coursera, Udemy, edX)**
- **Technical & Behavioral Interview Prep**

Hybrid Fallback System: If Gemini API free-tier rate limits (HTTP 429) are triggered, the application executes `get_role_based_fallback()`, pulling curated, high-quality guidelines directly from local dataset memory. This ensures **100% web application uptime**.

9. Experiment Tracking with MLflow

Model evaluation metrics were recorded on an 80/20 train-test split (**2,400 training samples, 601 test samples**):

Model Name	Accuracy	Precision	Recall	F1 Score	Evaluation Result
Random Forest	92.0%	91.0%	93.0%	92.0%	BEST MODEL (Deployed)
Logistic Regression	87.0%	86.0%	88.0%	87.0%	High Precision Baseline
Naive Bayes	81.0%	80.0%	83.0%	81.0%	Fast Baseline
AdaBoost	74.0%	73.0%	75.0%	74.0%	Boosting Baseline

Overfitting & Anomaly Verification

- **Training Set Accuracy:** 94.1%
- **Testing Set Accuracy:** 92.0%
- **Generalization Variance:** 2.1% (Confirms model is free of overfitting).

10. Deployment and Application

The project is deployed as an interactive web platform powered by **Flask 3.0**.

Technology Stack

Technology	Purpose
Python 3.14	Core programming language.
Flask 3.0	Web application framework and REST API routing.
Pandas / NumPy	Numerical processing and matrix transformations.
NLTK / SymSpell	Tokenization, stopword filtering, and lemmatization.
Scikit-Learn	TF-IDF vectorization, Random Forest classifier & regressor.
Google Generative AI	GenAI roadmaps, salary estimations, and interview prep.
ReportLab	On-the-fly dynamic PDF report generation.
Tailwind CSS / Plotly	Responsive frontend styling and interactive charts.
Docker / Docker Compose	Multi-stage containerized deployment.

11. Project Output / Demonstration

An end-to-end execution of a sample resume (`siddhi_resume.docx`) produced the following verified outputs:

Test Item	Observed Result
Uploaded File	<code>siddhi_resume.docx</code> (Word Document)
Selected Target Role	Machine Learning Engineer
Predicted Role (ML)	Software Engineer (92.0% Ensemble Confidence)
Overall ATS Score	78.75 / 100
Job Description Match	82.5% (Compatible)
Resume Strength	Strong (Regression Score: 81.2)
Target Role Display	Target Role: Machine Learning Engineer
Career Roadmap	6-Month ML Path (Python/Stats →→ Preprocessing →→ Classical ML →→ MLOps →→ Capstone)
Indian Salary Prediction	Entry: ₹6L - ₹9L Mid: ₹12L - ₹22L Senior: ₹25L - ₹45L
PDF Export	Downloaded <code>Resume_Analysis_Report.pdf</code> (6,555 bytes)

12. Testing and Expected Behavior

Test Case	Inputs	Expected Behavior	Observed Result
Standard PDF Upload	<code>sample_resume.pdf</code>	Extract text, run NLP, output ATS score & predictions	Pass (200 OK)
Standard DOCX Upload	<code>siddhi_resume.docx</code>	Extract docx text, score skills, render dashboard	Pass (200 OK)
Target Role Selection	Target: ML Engineer	Render 6-month roadmap & courses for ML Engineer	Pass (200 OK)
API Quota Exceeded (429)	Free Tier Rate Limit	Intercept error, trigger local fallback engine	Pass (200 OK - No Crash)
PDF Report Download	GET <code>/download_report</code>	Generate and stream multi-page ReportLab PDF	Pass (200 OK - Binary Stream)
Chart Data Endpoint	GET <code>/chart-data</code>	Return model accuracy & ATS distribution JSON	Pass (200 OK - Valid JSON)

13. Limitations and Future Scope

Limitations

- Language Scope:** Currently optimized for English-language resumes.
- Free Tier API Quotas:** Gemini API free-tier key restricts concurrent requests (handled gracefully by fallback engine).
- Template Layout Variance:** Non-standard multi-column resume graphic layouts may affect raw text extraction order.

Future Scope

- **Transformer Fine-Tuning:** Fine-tune domain-specific BERT / RoBERTa models for multi-label skill extraction.
- **Multilingual Support:** Integrate automated translation for non-English candidate resumes.
- **Real-Time Job API:** Connect with Live LinkedIn / Indeed job APIs for real-time opening recommendations.
- **Enterprise ATS Webhooks:** Expose REST APIs for seamless integration with enterprise HR platforms (Workday, Greenhouse, Lever).

14. Conclusion

The **ResumeAI** project successfully delivers an end-to-end, production-ready AI solution for candidate screening and career development. By combining deterministic NLP tokenization, TF-IDF vectorization, ensemble Random Forest classification, Explainable AI (XAI), and Google Gemini Generative AI with a resilient fallback mechanism, the platform achieves **92.0% role classification accuracy** while guaranteeing **100% system availability**. The system bridges automated candidate evaluation with actionable career roadmaps, empowering both recruiters and job seekers.

15. References

- Project Repository: <https://github.com/Siddhigandhi854/ResumeAI>
- Scikit-Learn Documentation — Random Forest Classifier & TF-IDF Vectorizer: <https://scikit-learn.org/>
- NLTK Documentation — Natural Language Toolkit for Python: <https://www.nltk.org/>
- Google Generative AI SDK — Gemini API Documentation: <https://ai.google.dev/>
- Flask Web Framework Documentation: <https://flask.palletsprojects.com/>
- ReportLab PDF Generation Library: <https://www.reportlab.com/>

— End of Project Report —